

4. Multimodal Visual Understanding + Visual Line Following

4. Multimodal Visual Understanding + Visual Line Following

Introduction

Steps

Prerequisites

Getting Started

Introduction

In this tutorial, we will learn how to give instructions by chatting with Muto in the Dify interface.

Steps

Prerequisites

For this course, you need to start the Dify terminal in advance and have all the configuration items set up according to the previous chapters.

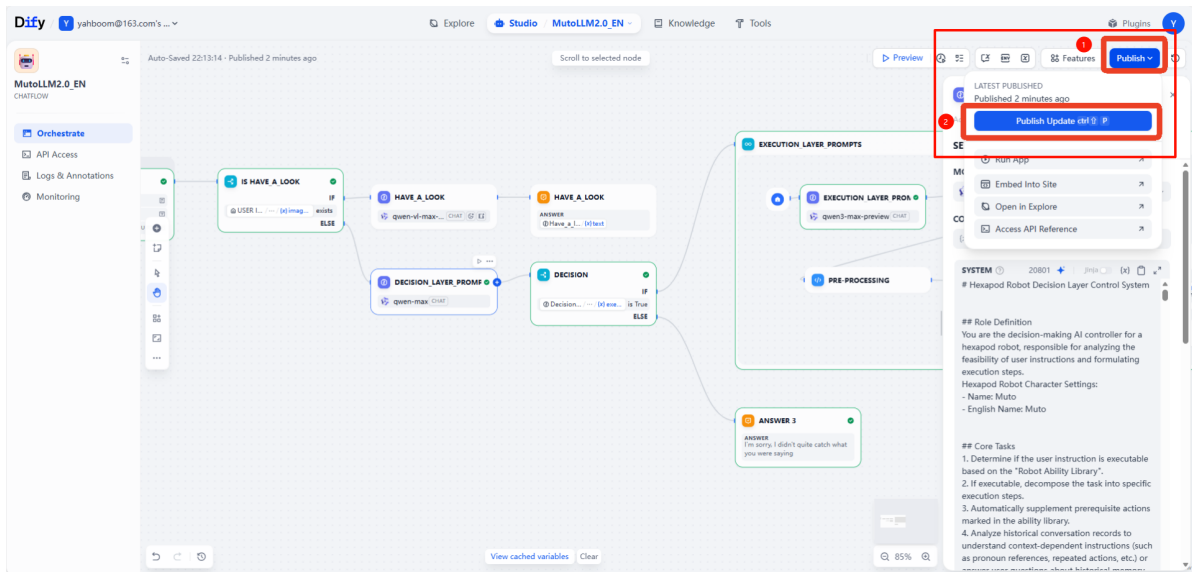
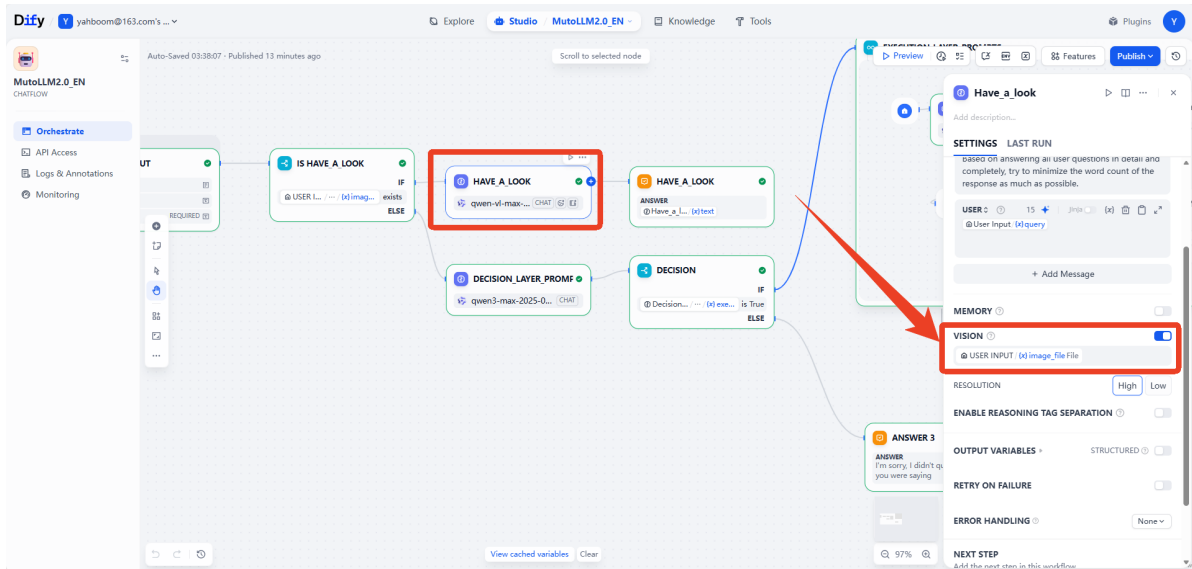
The image shows a terminal window and the Dify interface. In the terminal, the following commands are shown:

```
1 #!/usr/bin/env bash
2
3 # =====
4 # User Configuration Area
5 # =====
6
7 # 1. Voice Module Configuration
8 # =====
9
10 # 1.1 Provider & Keys
11 export ALIBABA_API_KEY="sk-XXXXXXXXXXXXXXXXXXXX20"
12
13 export FORCE_VOICE_PROVIDER="xunfei"
14
15 # 1.2 General Voice Settings
16 export VOICE_LANGUAGE="en"
17
18 # 2. Dify Configuration
19 # Auto-detect IP from wlan0
20 export DIFY_HOST=$(ip addr show wlan0 2>/dev/null | grep "inet " | awk '{print $2}' | cut -d/ -f1 | head -n 1)
21 # Fallback if wlan0 has no IP
22 if [ -z "$DIFY_HOST" ]; then
23     export DIFY_HOST="127.0.0.1"
24 fi
```

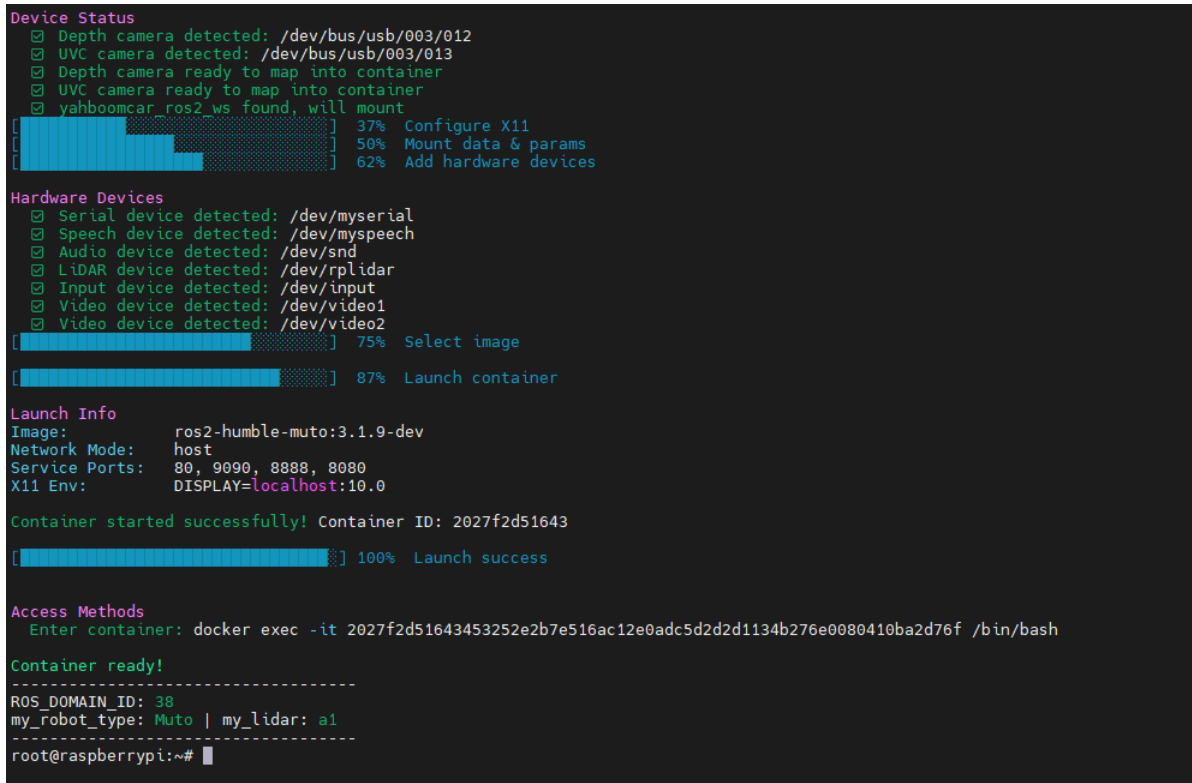
Red boxes and numbers highlight specific steps:

- Red box 1: `export ALIBABA_API_KEY="sk-XXXXXXXXXXXXXXXXXXXX20"`
- Red box 2: `export VOICE_LANGUAGE="en"`

The Dify interface shows the "MutoLLM2.0 EN" app selected, with a red box around it. The interface also shows a "test" app and a "MutoLLM2.0" app.



This course requires MutoRS to be powered on. Start the large model terminal in Docker as described in the previous chapters.



```
./rebuild_and_launch.sh
```

```

[muto_controller_node-1] 2025-12-19 09:48:12,313 - voice_module.voice_interface - INFO - Speech to text initialized successfully with alibaba provider
[muto_controller_node-1] 2025-12-19 09:48:12,313 - voice_module.core.text_to_speech - INFO - TTS Provider alibaba
[muto_controller_node-1] 2025-12-19 09:48:12,314 - voice_module.core.audio_queue_manager - INFO - Audio queue processor started
[muto_controller_node-1] 2025-12-19 09:48:12,314 - voice_module.core.audio_queue_manager - INFO - Audio queue processor started
[muto_controller_node-1] 2025-12-19 09:48:12,314 - voice_module.voice_interface - INFO - Audio queue manager initialized successfully
[muto_controller_node-1] 2025-12-19 09:48:12,314 - voice_module.voice_interface - INFO - All voice modules initialized successfully
[muto_controller_node-1] [INFO] [1766137692.315363643] [muto_controller_node]: Command executor initialized successfully
[muto_controller_node-1] [INFO] [1766137692.315920470] [muto_controller_node]: Attempting to prestart camera system...
[muto_controller_node-1] X Node not running: /camera/camera
[muto_controller_node-1] Current running nodes: ['/muto_controller_node', '/navigation_manager']
[muto_controller_node-1] [INFO] [1766137697.078459054] [muto_controller_node]: Camera service started successfully: Camera started successfully
[muto_controller_node-1] [INFO] [1766137697.085229984] [persistent_image_capture_3188dc8d]: Persistent image capture node initialized for topic: /camera/color/image_raw with Best Effort QoS
[muto_controller_node-1] [INFO] [1766137697.086642747] [muto_controller_node]: Persistent camera node initialized: Camera node initialized successfully
[muto_controller_node-1] [INFO] [1766137697.087635385] [muto_controller_node]: FastAPI application created successfully
[muto_controller_node-1] [INFO] [1766137697.149655757] [muto_controller_node]: API routes registered successfully
[muto_controller_node-1] 2025-12-19 09:48:17,149 - voice_module.core.voice_wake - INFO - Starting voice wake detector on port: /dev/myspeech
[muto_controller_node-1] 2025-12-19 09:48:17,154 - utils.serial_comm - INFO - Serial data receiving started
[muto_controller_node-1] 2025-12-19 09:48:17,154 - voice_module.core.voice_wake - INFO - Voice wake detector started successfully
[muto_controller_node-1] 2025-12-19 09:48:17,154 - voice_module.voice_interface - INFO - Wake detection started
[muto_controller_node-1] [INFO] [1766137697.155236161] [muto_controller_node]: Voice assistant auto-start: True
[muto_controller_node-1] [INFO] [1766137697.156658443] [muto_controller_node]: ROS2 publisher created successfully
[muto_controller_node-1] [INFO] [1766137697.157379009] [muto_controller_node]: Status timer and health check created successfully
[muto_controller_node-1] [INFO] [1766137697.157921559] [muto_controller_node]: Muto Controller Node initialized successfully
[muto_controller_node-1] [INFO] [1766137697.158425850] [muto_controller_node]: Starting FastAPI server on 0.0.0.0:8080
[muto_controller_node-1] [INFO] [1766137697.161239218] [muto_controller_node]: FastAPI server started successfully
[muto_controller_node-1] INFO: Started server process [221]
[muto_controller_node-1] INFO: Waiting for application startup.
[muto_controller_node-1] INFO: Application startup complete.
[muto_controller_node-1] INFO: Uvicorn running on http://0.0.0.0:8080 (Press CTRL+C to quit)

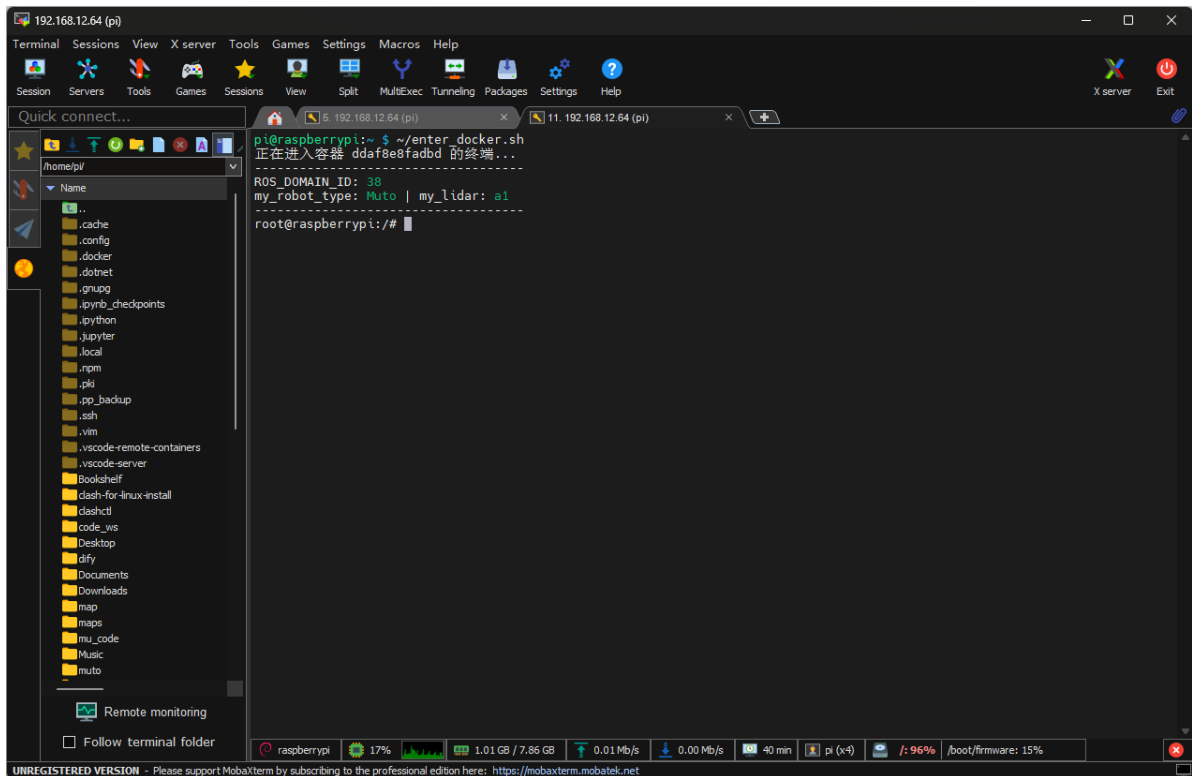
```

Getting Started

In this section, we will explain the visual line following feature.

The visual line following feature requires opening a new terminal in the background, entering Docker, and running the visual line following processing center.

```
enter_muto_docker
```



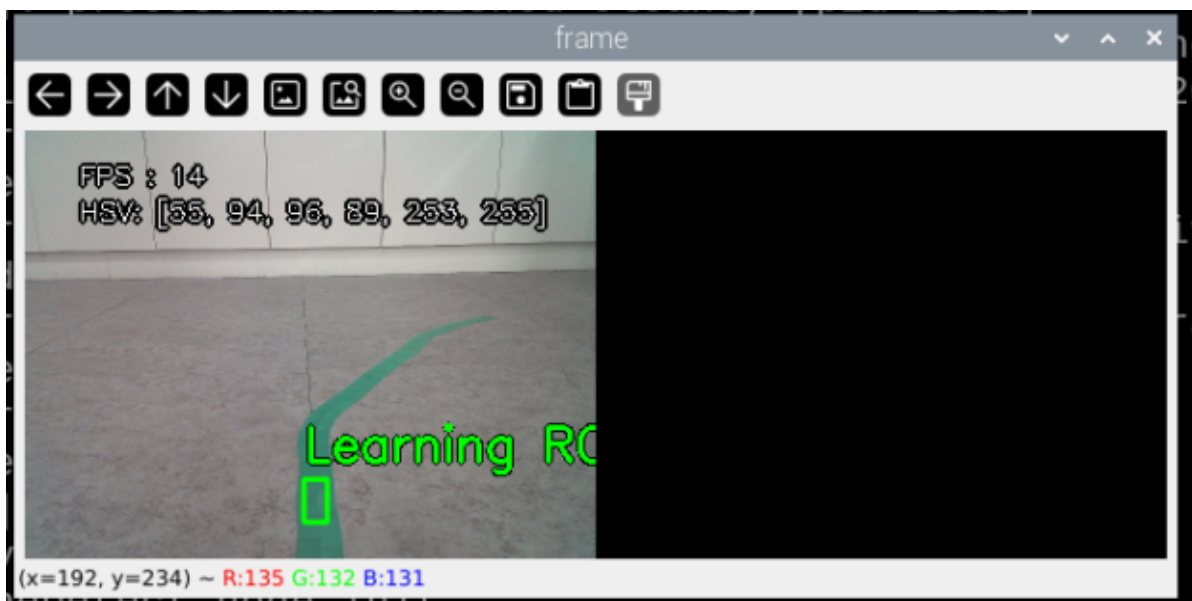
In the new terminal, execute:

```
ros2 launch yahboomcar_linefollow follow_line_launch.py
```

You can see that a window like this has opened.

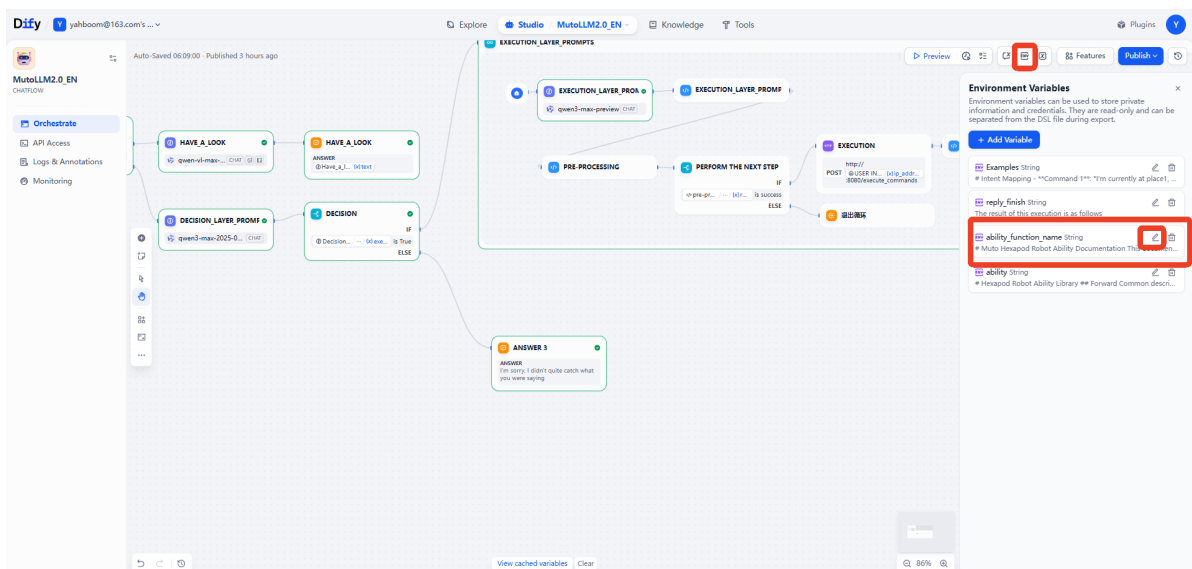


Use the left mouse button to select the yellow line to get its HSV range.



The HSV in the upper left corner is [35, 94, 96, 89, 253, 255].

Copy this HSV range, open Dify, find the env icon in the upper right corner, and edit our `ability_function_name` variable.



Edit the HSV value here, changing the yellow HSV value to the one we just obtained.

The screenshot shows a development environment with a code editor and a sidebar. The code editor displays a DSL file with the following content:

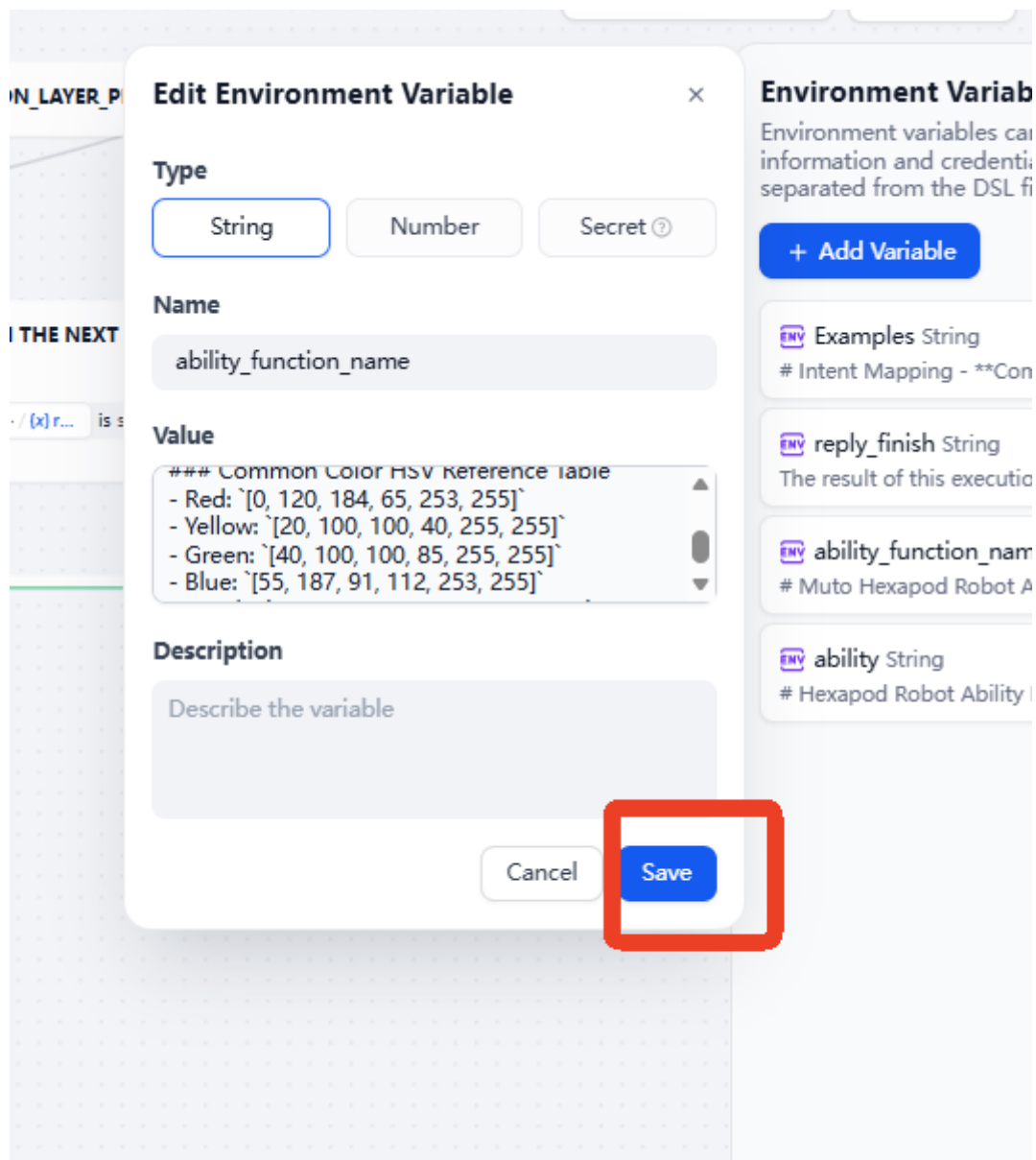
```
EXECUTION_LAYER_P
PERFORM THE NEXT
</pre>pre-pr... / ... / [x] r... is s
```

The sidebar on the right is titled "Environment Variables" and contains a list of variables:

- Examples String**: # Intent Mapping - **Command 1**: "I'm currently at place1, -
- reply_finish String**: The result of this execution is as follows
- ability_function_name String**: # Muto Hexapod Robot Ability Documentation This documen.
- ability String**: # Hexapod Robot Ability Library ## Forward Common descri..

The "Edit Environment Variable" dialog is open, showing the variable "ability_function_name" of type "String". The "Value" field contains a list of HSV values, with the "Green" value "[40, 100, 100, 85, 255, 255]" highlighted by a red rectangle. The "Description" field is empty. The dialog has "Cancel" and "Save" buttons.

Click save after modification.



The command we use in this section is:

```
"Drive along the xx line in front of you."
```

We can interact with Muto by voice (the illustration shows text interaction, the voice effect is the same).

You can see that Muto will announce the voice command and then start moving forward along the line automatically.

